

Apache Kafka Basics

Data Engineering Curriculum — Day 26 Notes

1. Streaming Concepts

In modern data engineering, data is continuously generated by various sources (e.g., IoT devices, user clicks, server logs, financial transactions). Traditional batch processing struggles to handle this continuous flow efficiently. This is where **stream processing** comes in.

- **Batch Processing:** Processes a large volume of data all at once at scheduled intervals (e.g., nightly ETL jobs). High latency but high throughput.
- **Stream Processing:** Processes data records continuously as they arrive. Low latency, enabling real-time analytics and fast reactions to events.
- **Event:** A fundamental entity in stream processing, representing a "thing that happened" along with its state at a specific point in time (e.g., "User A clicked Button B at 10:00 AM").

Why Apache Kafka? Kafka is a distributed event streaming platform capable of handling trillions of events a day. It provides high throughput, built-in partitioning, replication, and fault tolerance.

2. Topics

In Kafka, data is organized and categorized into **Topics**. You can think of a topic as a folder in a filesystem, and the events are the files in that folder.

- Topics are the core abstraction for publishing and subscribing to data.
- A topic is a continuous stream of events of a similar type.
- Topics in Kafka are *multi-producer* and *multi-subscriber*: a topic can have zero, one, or many producers writing to it, and zero, one, or many consumers reading from it.
- Data in a topic is retained for a configurable period (e.g., 7 days) regardless of whether it has been consumed.

3. Partitions

To scale a topic across multiple servers (brokers), Kafka breaks topics down into **Partitions**. This is the key to Kafka's scalability and parallel processing capabilities.

- A single topic can have one or many partitions distributed across the cluster.
- Each partition is an ordered, immutable sequence of records.
- **Offsets:** Every message within a partition gets an incremental ID called the *offset*. This offset uniquely identifies the message within that specific partition.
- Ordering is only guaranteed *within a specific partition*, not across the entire topic.

Partitioning Strategy: When a producer sends a message, it can specify a key. Kafka hashes this key to determine which partition the message belongs to. All messages with the same key are guaranteed to go to the same partition, ensuring ordered processing for specific entities (e.g., all events for a single User ID).

4. Producers

Producers are client applications that publish (write) events to Kafka topics.

- Producers automatically know which broker and partition to write to based on cluster metadata.
- They can choose to receive acknowledgment of data writes:
 - `acks=0`: Producer doesn't wait for acknowledgment (fastest, high data loss risk).
 - `acks=1`: Producer waits for the leader partition to acknowledge (balanced).
 - `acks=all`: Producer waits for the leader and all in-sync replicas to acknowledge (slowest, no data loss).

5. Consumers

Consumers are client applications that subscribe to (read) events from Kafka topics.

- Consumers read data in parallel by connecting to specific partitions.
- They read data in the order it was produced within a partition.

- **Consumer Groups:** Consumers typically organize themselves into Consumer Groups to share the workload. Each partition in a topic is assigned to exactly one consumer within the consumer group, allowing for massive parallel consumption and fault tolerance.
- Kafka uses the concept of consumer offsets to track which messages have been successfully processed, allowing consumers to resume from where they left off in case of failure.